

Project1 Final Write-up

一、概括针对测试版的改进以及改进动机

动机：

在测试版中，我们使用了提示的“三次翻转”算法。虽然其空间复杂度为 $O(1)$ ，且对于简单的比特旋转任务在实际运行中表现也尚可，但是之后简单地加上提示1、2的方法却反而在性能上有所下降。

在“计算机系统工程导论”这门课中学过，要想实现性能的优化，关键在于找到“瓶颈”，即限制性能提升的关键因素。该实验代码的核心瓶颈在于：算法对数组中的每一个 bit 都进行了 3 次读写操作，且每次操作都依赖

`bitarray_get` 和 `bitarray_set` 函数，引入了大量的函数调用开销、分支预测开销以及整数除法与取模运算，用于计算字节和位偏移。

改进：

为了打破单 bit 操作的性能瓶颈，我们在思考讨论后决定进行以下优化：

1. **宏观算法优化：** 采用 **Gries-Mills 块交换算法** 替代三次翻转，将不规则的旋转转化为一系列等长且互不重叠的块交换，大幅降低每个 bit 的平均移动次数。
2. **微观操作优化：** 引入了 `fast_bit_copy` 函数，彻底摒弃 `bitarray_get/set`。在处理连续位拷贝时，通过对齐检查，将操作粒度从 Bit 提升至 Byte。对于非对齐的内存块，采用 **位移拼接** 策略进行批量读写；对于完全对齐的块，直接利用底层优化过的 `memmove`。

二、性能分析与瓶颈定位（文字 + 截图）

先补充一张使用perf工具生成的原始代码的热力图，可以发现整个程序运行期间，CPU 有 99.94% 的时间都死死地卡在 `bitarray_rotate` 这一个函数里，这说明想要对程序的性能进行优化，除了优化这个函数，其他任何地方的微调都是毫无意义的。



在优化测试版代码之前，使用 macOS 系统自带的 `sample` 工具对测试版代码进行了热点分析，在终端中输入 `./everybit -l & sample $! 3` 在后台同步抓取程序运行时的进程调用栈，结果如下：

```
Call graph:
2536 Thread_29612525   DispatchQueue_1: com.apple.main-thread (serial)
2536 start (in dyld) + 6076 [0x19c7e2b98]
2536 main (in everybit) + 312 [0x104d189f8] main.c:0
  2170 timed_rotation (in everybit) + 232 [0x104d18d10] tests.c:291
  + 2170 bitarray_rotate (in everybit) + 392,280,... [0x104d18860,0x104d187f0,...] bitarray.c:201
  363 timed_rotation (in everybit) + 204 [0x104d18cf4] tests.c:287
  + 242 bitarray_randfill (in everybit) + 32 [0x104d186b4] bitarray.c:185
  + ! 169 rand (in libsystem_c.dylib) + 92,60 [0x19ca23e8c,0x19ca23e6c]
  + ! 73 DYLD-STUB$$rand (in everybit) + 4,8 [0x104d193b8,0x104d193bc]
  + 61 bitarray_randfill (in everybit) + 36,40 [0x104d186b8,0x104d186bc] bitarray.c:184
  + 60 bitarray_randfill (in everybit) + 28,32 [0x104d186b0,0x104d186b4] bitarray.c:185
  2 timed_rotation (in everybit) + 164 [0x104d18ccc] tests.c:287
  + 2 bitarray_free (in everybit) + 32 [0x104d1862c] bitarray.c:141
  + 2 _szone_free (in libsystem_malloc.dylib) + 700 [0x19c99b334]
  + 2 free_large (in libsystem_malloc.dylib) + 772 [0x19c98dc7c]
  + 2 madvise (in libsystem_kernel.dylib) + 8 [0x19cb44870]
  1 timed_rotation (in everybit) + 380 [0x104d18da4] tests.c:307
  1 printf (in libsystem_c.dylib) + 80 [0x19ca40b78]
  1 vfprintf_l (in libsystem_c.dylib) + 156 [0x19ca25a5c]
  1 __vfprintf (in libsystem_c.dylib) + 11400 [0x19ca186f0]
  1 __sfwrite (in libsystem_c.dylib) + 836 [0x19ca18cc0]
  1 __sflush (in libsystem_c.dylib) + 232 [0x19ca23d6c]
  1 _swrite (in libsystem_c.dylib) + 108 [0x19ca25ccc]
  1 _swrite (in libsystem_c.dylib) + 24 [0x19ca44d78]
  1 __write_nocancel (in libsystem_kernel.dylib) + 8 [0x19cb45aa4]

Total number in stack (recursive counted multiple, when >=5):

Sort by top of stack, same collapsed (when >= 5):
bitarray_rotate (in everybit)      2170
rand (in libsystem_c.dylib)        169
bitarray_randfill (in everybit)    121
DYLD-STUB$$rand (in everybit)      73
```

观察 Call Graph 可以发现在主线程的生命周期中，除了数据准备外，绝对的计算瓶颈全部集中在核心旋转逻辑上（上图中红框内的部分）。

在采样数据中，2179 个采样点全部聚集在 `bitarray_rotate` 这一层级，但是调用栈中并没有出现原本预期会被大量调用的底层工具函数，这说明在 `-O3` 优化级别下，Clang 编译器已经尽力消除了函数调用栈进出的开销，但是程序的吞吐量依然低下。

因此，想要在此基础上对代码进行进一步优化的唯一方法是抛弃逐位操作，在代码层面强制进行按字节或者按字对齐的批量操作，大幅度削减循环迭代的次数。

三、改进思路与方法（伪代码）

优化核心：

顶层使用块交换保证区间不重叠，底层使用 Shift-and-OR 进行高速字节拷贝。

伪代码 1: Gries-Mills 块交换旋转算法

Plaintext

```
1 function bitarray_rotate_left(bitarray, start, length, left_amount):
2     a_len = left_amount
3     b_len = length - left_amount
4     a_start = start
5     b_start = start + left_amount
6
7     while a_len > 0 and b_len > 0:
8         if a_len < b_len:
9             // 块A比块B短，将A与B的尾部等长部分交换
10            block_swap(bitarray, a_start, b_start + b_len - a_len, a_len)
11            b_len = b_len - a_len
12        else if a_len > b_len:
13            // 块A比块B长，将A的头部等长部分与B交换
14            block_swap(bitarray, a_start, b_start, b_len)
15            a_start = a_start + b_len
16            a_len = a_len - b_len
17        else:
18            // 长度相等，直接交换，旋转完成
19            block_swap(bitarray, a_start, b_start, a_len)
20            break
```

伪代码 2: Shift-and-OR

Plaintext

```
1 function fast_bit_copy(dest, dest_offset, src, src_offset, bit_len):
2     // 1. 处理头部不对齐的零碎位（逐位拷贝）
3     while src 未对齐到字节边界 且 bit_len > 0:
4         copy_single_bit()
5         bit_len--
6
7     // 2. 核心大块批量拷贝（进入字节级操作）
8     if bit_len ≥ 8:
9         byte_count = bit_len / 8
10        if dest 也恰好对齐到字节边界:
11            memmove(dest, src, byte_count) // 完美对齐，调用底层库
12        else:
```

```

13         // Shift-and-OR: 跨字节拼接写入
14         right_shift_amount = dest_offset % 8
15         left_shift_amount = 8 - right_shift_amount
16         for i from 0 to byte_count - 1:
17             src_byte = src[i]
18             // 将源字节撕裂, 分别拼接到目标内存的两个相邻字节中
19             dest[d_byte] = (dest[d_byte] & mask1) | (src_byte <<
right_shift_amount)
20             dest[d_byte + 1] = (dest[d_byte + 1] & mask2) | (src_byte >>
left_shift_amount)
21
22         bit_len = bit_len % 8 // 剩余不足一字节的位
23
24     // 3. 处理尾部零碎位
25     while bit_len > 0:
26         copy_single_bit()
27         bit_len--

```

四、编程过程中遇到的 Bug 或陷阱

字节序与位移方向的错乱

在实现 Shift-and-OR 拼接时, 很难直观地判断到底是该 `<<` 还是 `>>`。由于位数组在字节内的存储顺序通常是从低位到高位, 当试图把一个 bit 向高索引移动时, 实际上在单个字节内部可能是向高位位移 (Left Shift `<<`)。掩码的取反操作 (`~mask`) 也极易算错, 导致覆盖了目标字节中原本不属于本次修改区间的相邻 bit。

尾部边界越界

在处理最后剩余的 `bit_len % 8` 个尾部零散 bit 时, 循环边界条件稍有不慎, 就会多写 1 个 bit, 破坏了相邻的其他数据结构, 导致 Segment Fault 或者是断言失败。

五、其他信息：对性能没有提升的“无用功”

基于“三次翻转”算法进行优化时, 曾尝试过以下提示中的两种思路, 但没有明显的效果, 甚至性能反而略有下降:

尝试 1: 引入中间数组进行块转移

- **思路:** 将需要旋转的左侧块 `a` 复制到临时数组 `tmp` 中, 把右侧块 `b` 平移到最终位置, 再从 `tmp` 把 `a` 填回。
- **无用原因:** 动态内存分配带来了巨大的系统调用开销; 而且如果底层依旧使用单 bit 读写 (`get/set`), 算术运算的指令瓶颈依然存在, 整体性能不升反降。

尝试 2: 单步循环移位法

- **思路:** 编写一个仅左移 1 位的函数, 取旋转次数与数组长度的模数作为循环次数, 多次重复调用单步移位。
- **无用原因:** 引发了时间复杂度退化。将原本 $O(N)$ 的操作拖成了 $O(N \times K)$ (K 为移位次数)。在面临大数组负载时, 计算量将呈二次方级激增。

总结教训：

这两次弯路让我明确了底线：在底层系统编程中，绝不能以引入动态内存分配开销，或牺牲算法时间复杂度为代价，去进行所谓的“逻辑简化”。

六、最终性能表现

经过上述从算法到内存布局的深度重构，程序的性能实现了质的飞跃。

- 优化前表现： 在限定时间内，测试版代码大约能跑到 38 轮。

```
Tier 32 (~1MB) completed in 0.043416s
Tier 33 (~2MB) completed in 0.070050s
Tier 34 (~4MB) completed in 0.112608s
Tier 35 (~7MB) completed in 0.182188s
Tier 36 (~12MB) completed in 0.297154s
Tier 37 (~19MB) completed in 0.484129s
Tier 38 (~31MB) completed in 0.783528s
Tier 39 (~51MB) exceeded 1.00s cutoff with time 1.266074s
Successfully completed tier: 38
---- END RESULTS ----
```

- 优化后表现： 在完全相同的测试环境下，最终版代码达到了 43 轮。

```
Tier 37 (~19MB) completed in 0.042267s
Tier 38 (~31MB) completed in 0.060953s
Tier 39 (~51MB) completed in 0.091998s
Tier 40 (~83MB) completed in 0.109680s
Tier 41 (~135MB) completed in 0.272916s
Tier 42 (~218MB) completed in 0.466617s
Tier 43 (~354MB) completed in 0.764118s
Tier 44 (~573MB) exceeded 1.00s cutoff with time 1.186915s
Successfully completed tier: 43
---- END RESULTS ----
```

这一显著的吞吐量提升，证明了消除单位存取开销、充分利用 CPU 字节级处理能力以及利用 L1 栈缓存进行数据交换的有效性。